

Objetivo: Compreender os conceitos básicos de POO e os pilares de Encapsulamento e Herança.

Introdução à POO e seus Benefícios

O que é POO? (Conceito de objetos, classes, atributos e métodos).

Por que usar POO? (Reutilização de código, organização, manutenção, modelagem do mundo real).

Dart como Linguagem Orientada a Objetos: Breve menção de que "Tudo é objeto" em Dart e sua natureza multiparadigma

Classes e Objetos em Dart

Declaração de Classes: Como definir uma classe em Dart (ex: `class Pessoa { ... }`).

Instanciação de Objetos: Criando instâncias de classes (ex: `Pessoa p = Pessoa('Nome', 30);`).

Construtores: Construtor padrão e construtores nomeados.

Palavra late: Entendimento do seu uso para inicialização tardia e null safety .

Encapsulamento

Conceito: Proteção de dados e controle de acesso.

Encapsulamento em Dart: Uso do `_` (underline) para atributos e métodos privados .

Gets e Sets: A flexibilidade de Dart na criação de getters e setters.

Herança

Conceito: Reutilização de comportamentos e hierarquia de classes.

Herança em Dart: Uso da palavra `extends` e `super` para chamar construtores ancestrais.

Sobrescrita (`@override`): Redefinindo comportamentos de classes ancestrais (ex: `toString()`).

Polimorfismo, Abstração e Conceitos Avançados

Objetivo: Aprofundar nos pilares de Polimorfismo e Abstração, e explorar Mixins e Métodos Estáticos.

Polimorfismo

Conceito: Capacidade de objetos de diferentes classes responderem ao mesmo método de maneiras distintas.

Polimorfismo por Sobrescrita: Revisão do toString() como exemplo prático

Polimorfismo por Herança: Como uma variável de um tipo ancestral pode referenciar objetos de classes descendentes.

Abstração

Conceito: Foco no "o quê" um objeto faz, em vez do "como".

Classes Abstratas: Definição e impossibilidade de instanciação.

Métodos Abstratos: Métodos sem implementação que obrigam as classes concretas a implementá-los.

Mixins

Conceito: Reutilização de código sem herança múltipla.

Uso de Mixins em Dart: Como adicionar métodos a classes usando with.

mixin vs. mixin class: Diferenças e casos de uso.

Métodos Estáticos e Palavras-chave Complementares

Métodos Estáticos: Métodos que pertencem à classe e não a uma instância específica.

const vs. final: Diferenças e quando usar cada um.

Operadores ! e ?: Breve revisão de null safety